

THE TESTING ACADEMY · PLAYWRIGHT PRACTICE

XPath Master Cheat Sheet: Functions, Axes, Operators, Examples

Everything XPath can do for an automation tester on one page: the anatomy, **every function** you will actually use, **all 13 axes** with a family-tree diagram, operators, the indexing gotchas, dynamic-element patterns, and table strategies, with copy-paste examples for **Selenium and Playwright**.

20+ functions

All 13 axes + diagram

Dynamic element patterns

Table strategies

Selenium + Playwright

ON THIS PAGE

1. Anatomy of an XPath
2. Quick reference: the basics
3. Text matching
4. All the functions
5. All 13 axes + diagram
6. Operators
7. Indexing and its gotcha
8. Dynamic element patterns
9. Table strategies
10. Selenium and Playwright usage
11. XPath vs CSS
12. FAQ

1. Anatomy of an XPath

```
//tag[@attribute='value'][position()]/axis::tag
```

```
| | | | |
| | | | | — walk to related nodes (parent::, followi
| | | | | — pick one from the matched set ([1], [last()])
| | | — predicate: the condition in [ ] (@id='x', text()='y', contains(...))
| — the element tag, or * for any
| — // = search at ANY depth / = direct child only
```

- **Absolute** paths start at the root: `/html/body/div[2]/form/input` . One DOM change breaks them. Never ship one.
- **Relative** paths start with `//` and anchor on something stable:
`//input[@name='email']` . This is the default.

2. Quick reference: the basics

EXPRESSION	MEANING	EXAMPLE
<code>//tag</code>	Every tag, any depth	<code>//button</code>
<code>//*</code>	Any element	<code>//*[@data-testid='save']</code>
<code>//tag[@attr='v']</code>	Attribute equals	<code>//input[@name='email']</code>
<code>//tag[@attr]</code>	Attribute exists	<code>//img[@alt]</code>
<code>//tag[@a='x' and @b='y']</code>	Multiple conditions	<code>//input[@type='text' and @required]</code>
<code>//A//B</code>	B anywhere under A	<code>//form//button</code>
<code>//A/B</code>	B direct child of A	<code>//ul/li</code>
<code>//A //B</code>	Union of two sets	<code>//input //textarea</code>
<code>.</code> and <code>..</code>	Current node, parent	<code>//td[.='Total']/..</code> the row of that cell

3. Text matching

EXPRESSION	MATCHES	WATCH OUT
<code>//td[text()='Delivered']</code>	Exact own-text match	Fails on stray whitespace or nested spans

<code>//td[contains(text(),'Deliv')]</code>	Substring of own text	Also matches "Not Delivered": exactness is your job
<code>//td[.='Delivered']</code>	Full string value incl. children	Use when the text sits inside a nested span
<code>//td[normalize-space()='Delivered']</code>	Trimmed, whitespace-collapsed	The safe default for exact text
<code>//td[contains(normalize-space(),'Delivered')]</code>	Trimmed substring, incl. children	The safe default for partial text

Rule of thumb. `text()` reads only the node's own text; `.` reads everything inside it. A cell rendered as `<td><span>Delivered</span></td>` has EMPTY `text()` : that is the number one "my XPath works in theory" bug. When in doubt, use `normalize-space(.)` .

4. All the functions you will actually use

FUNCTION	WHAT IT DOES	EXAMPLE
<code>contains(a, b)</code>	Substring test	<code>//div[contains(@class, 'error')]</code>
<code>starts-with(a, b)</code>	Prefix test, great for dynamic ids	<code>//input[starts-with(@id, 'user_')]</code>
<code>text()</code>	The node's own text	<code>//a[text()='Login']</code>
<code>normalize-space(s)</code>	Trim + collapse whitespace	<code>//h2[normalize-space()='My Orders']</code>

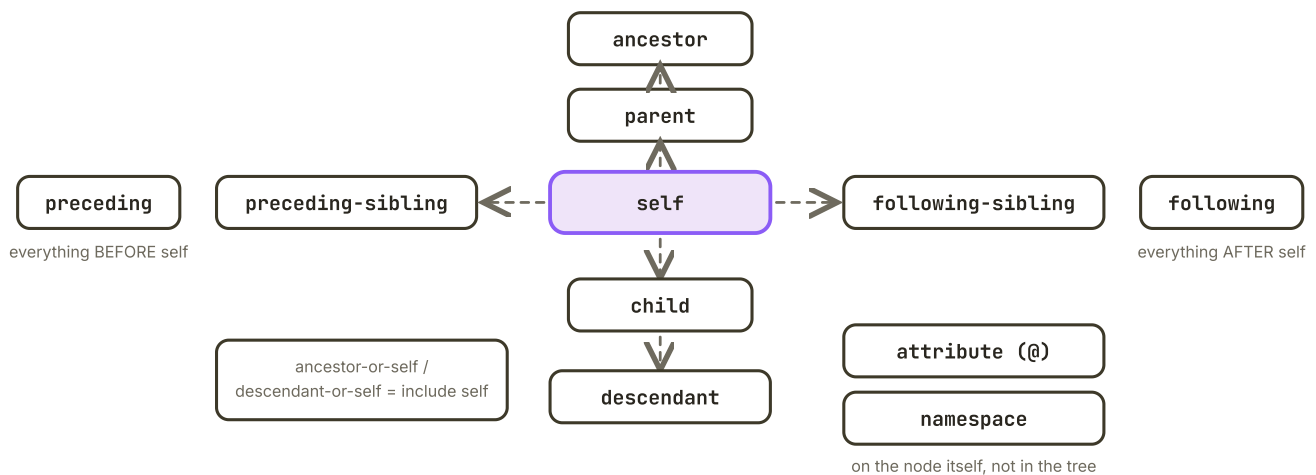
<code>translate(s, from, to)</code>	Character mapping: the case-insensitive trick	<code>//a[contains(translate(text(),'ABCDEFGHIJ</code>
<code>substring(s, start, len)</code>	Slice a string (1-based)	<code>substring(@id, 1, 4)</code>
<code>substring-before(s, d) / substring-after(s, d)</code>	Split around a delimiter	<code>substring-after(@href, '=') a query value</code>
<code>string-length(s)</code>	Length	<code>//td[string-length(text()) > 0] non-emp</code>
<code>concat(a, b, ...)</code>	Join strings; the quote-escape trick	<code>//td[text()=concat("It", "'", "s done")]</code>
<code>count(nodes)</code>	How many matches	<code>//tr[count(td)=5] rows with exactly 5 cells</code>
<code>position()</code>	Index within the current set	<code>//tr[position() ≤ 3] first three rows</code>
<code>last()</code>	The final index	<code>//tr[last()] , //tr[last()-1]</code>
<code>not(expr)</code>	Negation	<code>//tr[not(th)] data rows only</code>
<code>name() / local-name()</code>	The tag name (local-name ignores namespaces: SVG!)	<code>//*[local-name()='svg']//*[local-name()='</code>
<code>number(s) / boolean(x)</code>	Type casts for comparisons	<code>//td[number(text()) > 100]</code>

Browser XPath is version 1.0. `ends-with()` , `matches()` , `lower-case()` are XPath 2.0+: they do NOT work in Selenium or browsers. Emulate ends-with:

`//input[substring(@id, string-length(@id) - 5) = '_email']` , and lower-case with the `translate()` trick above.

5. All 13 axes, with the family tree

Axes are XPath's superpower over CSS: from any node you can walk in every direction. Read them as `axis::node` , "from here, go that way".



Stand on any node and you can walk the whole tree. CSS only ever walks down.

AXIS	SELECTS	REAL EXAMPLE
<code>self::</code>	The node itself	<code>//td[self::td]</code> (mostly used in checks)
<code>parent::</code>	The one direct parent (same as <code>..</code>)	<code>//input[@id='q']/parent::div</code>
<code>child::</code>	Direct children (the default axis)	<code>//tr/child::td = //tr/td</code>
<code>ancestor::</code>	All parents up to the root	<code>//span[text()='Anita']/ancestor::tr</code> the row containing a deep cell

ancestor-or-self::	Ancestors + the node	<code>//td/ancestor-or-self::*[@data-row]</code>
descendant::	Everything inside, any depth	<code>//tr[descendant::button[@title='Delete']]</code>
descendant-or-self::	Inside + the node (what // expands to)	<code>//table/descendant-or-self::td</code>
following-sibling::	Later siblings, same parent	<code>//label[text()='Email']/following-sibling::input</code>
preceding-sibling::	Earlier siblings, same parent	<code>//input[@id='q']/preceding-sibling::label</code>
following::	Every node after self in the document	<code>//h2[text()='Orders']/following::table[1]</code> the first table after a heading
preceding::	Every node before self	<code>//table/preceding::h2[1]</code> the nearest heading above
attribute::	Attributes (same as @)	<code>//input/attribute::name = //input/@name</code>
namespace::	Namespace nodes (XML; rare on the web)	You will likely never write this one

The two that pay rent: `ancestor::` to climb from a unique deep anchor (a cell's text) up to its row, and `following-sibling::` to hop from a label to its field. Master those two and 90% of "impossible" locators fall.

6. Operators

OPERATOR	MEANING	EXAMPLE
and / or	Combine predicates	<code>//input[@type='text' or @type='email']</code>
not()	Negate (a function, not a keyword)	<code>//tr[not(contains(@class, 'header'))]</code>
= / ≠	Equality	<code>//td[@colspan≠'2']</code>
> ≥ < ≤	Numeric comparison	<code>//tr[position() > 1]</code> skip the header
+ - * div mod	Arithmetic (div, not /)	<code>//tr[position() mod 2 = 0]</code> even rows
 	Union of node sets	<code>//button[@type='submit'] //input[@type='submit']</code>

7. Indexing, and the gotcha that breaks everyone once

```
// XPath counts from 1, not 0
//table//tr[1]           // first row
//table//tr[last()]      // last row
//table//tr[last()-1]    // second to last
//table//tr[position() ≤ 3] // first three

// THE GOTCHA: [n] binds to the nearest step, not the whole expression
//div/a[2]           // the 2nd link INSIDE EACH div (can match many!)
((//div/a)[2])       // the 2nd link of ALL matched links (exactly one)
```

If an index "randomly" matches multiple elements, you hit the binding gotcha: wrap the whole expression in parentheses before indexing.

`((//a[contains(@href, 'download')])[3])` is the third such link on the page, full stop.

8. Dynamic element patterns

PROBLEM	PATTERN
Id changes per build: <code>input-8347-email</code>	<code>//input[starts-with(@id,'input-') and contains(@id,'-email')]</code>
Class soup from frameworks	Anchor on stable attrs: <code>//*[@data-testid='row-actions']</code>
Text with an apostrophe	<code>//td[text()=concat("0", "'", "Brien")]</code>
Case differs across environments	<code>translate()</code> lower-case trick from the functions table
SVG icons (namespaced)	<code>//*[@local-name()='svg' and @data-icon='trash']</code>
Element identified only by its neighbor	<code>//label[normalize-space()='Email']/following-sibling::input[1]</code>
Nth card with some property	<code>(//div[contains(@class,'card')][.//span[text()='Active']])[2]</code>

9. Table strategies, end to end

Same demo table as the [CSS sheet](#), so you can compare the two languages line by line:

Order	Customer	Status	Action
#1001	Pramod	Delivered	<button>Delete</button>
#1002	Anita	Pending	<button>Delete</button>
#1003	Rahul	Delivered	<button>Delete</button>

```
// The row containing Anita (cell text anchors the row)
//tr[td[normalize-space()='Anita']]
```

```
// Anita's STATUS (3rd cell of that row)
//tr[td[normalize-space()='Anita']]/td[3]
```



```
// Click Delete on Anita's row
//tr[td[normalize-space()='Anita']]//button

// Same, but climbing UP from the anchor instead (ancestor axis)
//td[normalize-space()='Anita']/ancestor::tr//button

// Column by HEADER NAME, not position: Status cell of Anita's row
//tr[td[.='Anita']]//td[count(//th[normalize-space()='Status']/preceding-sibling::

// Second Delivered row (parentheses first, then index)
(//tr[td[normalize-space()='Delivered']])[2]

// All data rows, skipping the header row
//tr[not(th)]
```

The header-name trick explained.

`count(//th[.='Status']/preceding-sibling::th)+1` computes the column NUMBER of the Status header (count of headers before it, plus one), then `td[that number]` picks the same column in the data row. Reorder the columns and the locator still works: that is the payoff over `td[3]`.

10. Using XPath in Selenium and Playwright

```
# Selenium (Python)
driver.find_element(By.XPATH, "//tr[td[normalize-space()='Anita']]//button").click

// Selenium (Java)
driver.findElement(By.xpath("//label[text()='Email']/following-sibling::input")).

// Playwright (TypeScript) - xpath= prefix, or // is auto-detected
await page.locator("xpath=//tr[td[normalize-space()='Anita']]//td[3]").innerText()
await page.locator("//h2[text()='Orders']/following::table[1]").screenshot();

// DevTools console - test any XPath live
$x("//tr[td[normalize-space()='Anita']]")
```

Test before you commit. `$x("...")` in the browser console evaluates any XPath instantly. In Playwright, prefer user-facing locators (`getByRole` , `getByText`) where

they work; reach for XPath when you need axes: climbing to ancestors or hopping siblings.

11. XPath vs CSS: when to use which

NEED	CSS	XPATH
Id / class / attribute	Winner: shorter, faster to read	Works, wordier
Text matching (portable)	Not in standard CSS	Winner: <code>text()</code> , <code>contains()</code> , <code>normalize-space()</code>
Walk UP the tree	Only <code>:has()</code>	Winner: <code>parent::</code> , <code>ancestor::</code>
Sibling hops	<code>+</code> and <code>~</code> (forward only)	Winner: both directions, with conditions
Column by header name	Not expressible	Winner: the <code>count(th/preceding-sibling)</code> trick
Playwright text search	<code>:has-text()</code> etc. (Playwright-only)	Portable everywhere

12. FAQ

What is the difference between / and // ?

`/` steps to a direct child (from the root when leading); `//` searches any depth.
`//input` finds every input on the page; `/html/body/input` only a direct child of body.

Does XPath start at 0 or 1?

1. `//tr[1]` is the first row. And remember the binding gotcha: `((//a)[2]` vs `//div/a[2]` are different questions.

How do I select by text?

`//tag[normalize-space()='Exact']` for exact,
`//tag[contains(normalize-space(.), 'part')]` for partial. Avoid bare `text()='` on real apps: whitespace and nested spans break it.

Which axes should I learn first?

`ancestor::` and `following-sibling::` . They solve the two everyday problems: climb from a unique cell to its row, and hop from a label to its input.

Drill these on real markup in the [tables practice page](#) and the [Playwright practice library](#), or let [SnapLocator](#) generate and heal locators for you.

Pair it with the [CSS Selector Cheat Sheet](#) →

LEARN THIS LIVE WITH THE TESTING ACADEMY

PLAYWRIGHT COURSE

Playwright Automation Mastery

Master end-to-end Playwright: framework design, API testing, CI, and scale.

[Enroll Now →](#)

AI TESTER BLUEPRINT BATCH

AI Powered Testing Mastery

Become an AI-powered SDET. MCP, RAG, agents, and real projects, live with mentorship.

[Enroll in the Batch →](#)

Built by [The Testing Academy](#) · part of the [Playwright practice library](#).

Ready to go from reading to shipping? Join a live cohort at The Testing Academy.

 [Enroll Now](#)

 [AI Powered Testing Mastery](#)

[Browse all Playwright practice guides →](#)